

APPLIED RESEARCH

Experimental Validation of Translational Single LiDAR 2D Mechanism Into Autonomous Navigation and 3D SLAM of Mobile Robots

MARJORIE A. LEÓN RAMOS^{ID}, RAUL PAOLO RIVERA VERA^{ID},
DANIEL D. YANYACHI^{ID}, (Member, IEEE), ALFREDO MAMANI SAICO^{ID}, (Member, IEEE),
GERMAN ALBERTO ECHAIZ ESPINOZA^{ID}, JUAN CARLOS CUTIPA LUQUE, (Member, IEEE),
AND PABLO R. YANYACHI^{ID}, (Senior Member, IEEE)

Universidad Nacional de San Agustín de Arequipa, Arequipa 04000, Peru

Corresponding author: Daniel D. Yanyachi (dyanyachi@unsa.edu.pe)

This work was supported by the Universidad Nacional de San Agustín de Arequipa (UNSA) under Contract PI-018-2023-UNSA.

ABSTRACT Most autonomous navigation approaches for mobile robots rely either on 3D LiDAR sensors, which are often costly, or on horizontally mounted 2D LiDAR sensors, which do not support three-dimensional mapping. This paper presents the implementation of a 3D Simultaneous Localization and Mapping system integrated with autonomous navigation in a mobile robot using a Translational Single LiDAR 2D (TSL-2D) configuration: a three-dimensional mapping mechanism based on a single vertically mounted 2D LiDAR sensor. To verify the feasibility of the proposed approach, the methodology combines the mathematical foundations, implementation and evaluation of global planning algorithms—Dijkstra, A-star (A*), and Rapidly-exploring Random Tree Star (RRT*)—within the Robot Operating System 2 (ROS 2) framework, integrating point cloud generation from vertical LiDAR sweeps. Experiments are conducted both in simulated environments using Gazebo and in real-world scenarios. The qualitative results demonstrate that the system is reliable for evaluating autonomous navigation algorithms, including local and motion planners. Quantitatively, as a specific outcome of the proposed system, RRT* produces smoother and shorter trajectories compared to A* and Dijkstra, achieving a 7.62% reduction in path length. Furthermore, in comparison to RGB-D camera-based systems reported in the literature, the TSL-2D mechanism delivers superior precision in generating 2D projected maps from the acquired point clouds.

INDEX TERMS Autonomous, SLAM, LiDAR, mobile robotics, mapping, navigation.

I. INTRODUCTION

Simultaneous Localization and Mapping (SLAM) [1] is a core computational technique that enables a mobile robot to construct a map of an unknown environment while concurrently estimating its position within it. As part of autonomous navigation, numerous SLAM methodologies have been proposed, each with trade-offs in terms of accuracy, cost, and environmental adaptability.

The associate editor coordinating the review of this manuscript and approving it for publication was Zheng H. Zhu^{ID}.

Traditional 2D LiDAR-based SLAM approaches are widely adopted for their simplicity and computational efficiency; however, they often rely on additional RGB-D sensors to improve 3D depth perception [2]. Vision-based SLAM methods are also popular due to their lower cost and easier installation compared to LiDAR systems, as highlighted in [3]. For instance, [4] employs shore segmentation using Otsu's method to filter water and sky regions, while [5] introduces an innovative algorithm for accurately distinguishing moving objects from static ones. However, despite these advances, visual SLAM remains highly sensitive to lighting conditions, which can hinder its performance in poorly lit

or outdoor environments. In contrast, our approach—based on a 2D LiDAR sensor mounted in an unconventional configuration—ensures consistent performance regardless of lighting, making it more suitable for diverse environments, especially where visual information is unreliable.

On the other hand, 3D LiDAR-based systems [6], [7] offer high-resolution and accurate 3D mapping, and have also been widely applied in various SLAM approaches, as shown in comparative evaluations of LiDAR and IMU-based methods for 3D robotic mapping [8], but their adoption is frequently hindered by the high cost and power consumption of such sensors. Recent efforts have explored low-cost 3D LiDAR alternatives [9], aiming to balance performance with affordability, though challenges remain in precision and robustness. Additionally, precision comparisons between robotic platforms and handheld mechanisms [10] have shown that robotic platforms outperform handheld systems in terms of noise level and point distribution, suggesting greater efficiency for autonomous 3D mapping tasks.

As a response to these limitations, the Translational Single LiDAR 2D (TSL-2D) mechanism has emerged as an under-explored low-cost solution in three-dimensional mapping for autonomous navigation of mobile robotics [11]. This method leverages a vertically mounted 2D LiDAR sensor and combines its scan data with motion estimation—derived from the robot's translational and rotational movements via odometry—to reconstruct 3D environments, also referred to as push-broom 2D LiDAR in [12].

The applicability and versatility of the TSL-2D approach have been demonstrated in a variety of domains. These include anomaly detection within tubular structures for hypersonic transportation systems [13], to capture a dense view of the surrounds in road-vehicle localization [14], [15], in 3-D loop-closure detection with small UAVs in large-scale outdoor environments [16], [17], non-destructive estimation of leaf area in agricultural monitoring [18], forest environment modeling [19], and cost-effective autonomous navigation research [20]. Moreover, its adaptability to diverse surface types and materials further extends its potential for deployment in heterogeneous environments [21].

Therefore, to enable autonomous navigation using the TSL-2D mechanism, it is essential to integrate trajectory planning algorithms. These algorithms compute feasible paths that guide a robot from a starting point to a target location while avoiding obstacles and satisfying specific constraints such as safety, energy efficiency, or time optimization [22], [23], based on the environment mapped through TSL-2D.

The main aims and contributions are outlined as follows:

- To demonstrate the feasibility of the TSL-2D mechanism for validating autonomous navigation applications and algorithms, addressing the challenges identified in a previous study [11], which focused solely on evaluating the accuracy of 3D mapping.
- To implement the TSL-2D mechanism both in simulation and in a real-world setup, as illustrated in Figure 1.

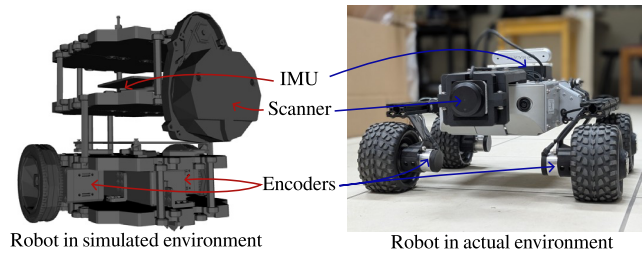


FIGURE 1. Minimum sensor requirements for the TSL-2D configuration, proposed as an alternative to RGB-D, vision-based, and 3D LiDAR-based SLAM approaches reported in the literature.

- To compare and evaluate widely recognized global path planning algorithms—Dijkstra, A*, and RRT*—using performance metrics such as trajectory smoothness, deviation, and positional error within the three-dimensional environment reconstructed by the SLAM system.
- To compare and evaluate the accuracy of the 2D map projected from the point cloud generated by the proposed TSL-2D mechanism against a widely used RGB-D-based approach.

It is important to note that this study focuses exclusively on global path planning for feasibility verification of the TSL-2D mechanism and does not address other components of autonomous navigation, such as local planning or additional algorithmic modules. Nevertheless, the proposed system is fully compatible with the integration and assessment of further navigation algorithms, which is suggested as a direction for future work.

II. METHODOLOGY

In this section, we describe the fundamentals of the path planning algorithms used, as well as the experimental setup and mission design employed to compare the algorithms. As a preliminary overview, the proposed mechanism has been implemented both in simulation using the TurtleBot3¹ simulator and on a real robot, leveraging Nav2² from ROS 2, an open-source framework for autonomous navigation of mobile robots, detailed in II-B.

In general, global planning strategies are categorized by their underlying approach and area of application. Grid-based methods [24], such as A* and Dijkstra, operate on a discretized representation of the environment. Sampling-based methods [25], including Rapidly-exploring Random Trees (RRT) and Probabilistic Roadmaps (PRM), explore the robot's configuration space using randomly sampled points. Hybrid approaches [26], [27] combine conventional planning techniques with optimization algorithms or machine learning to address challenges in complex or dynamic environments.

¹<https://emanual.robotis.com/docs/en/platform/turtlebot3/simulation/>

²<https://docs.nav2.org/>

A. FUNDAMENTALS OF PATH PLANNING ALGORITHMS

1) DIJKSTRA ALGORITHM

The Dijkstra algorithm, proposed by E. Dijkstra in [28], is used to find the shortest path in a directed and weighted graph. Let $G = (V, E)$ be a graph where V represents the set of vertices, and E denotes the set of edges with non-negative weights $w(u, v) \geq 0$ for $u, v \in V$, as shown in Figure 2. The goal is to determine the shortest distance from a source node s to all nodes $v \in V$, by minimizing the cumulative cost function, as expressed in (1):

$$d[v] = \min(d[v], d[u] + w(u, v)) \quad (1)$$

where $d[v]$ represents the minimum accumulated distance from s to v , and $d[u]$ is the distance from s to u , a previously visited neighboring node.

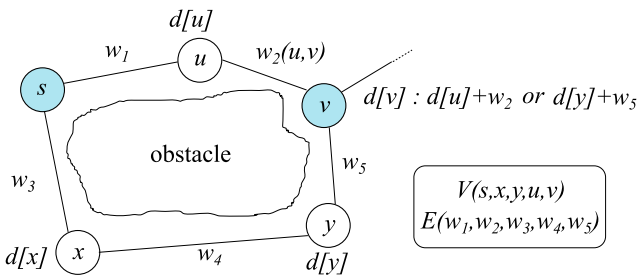


FIGURE 2. Example of a weighted graph used to illustrate Dijkstra's algorithm for shortest path computation.

In Nav2, this algorithm is implemented in the function `calcNavFnDijkstra`. The distances are initialized as $d[s] = 0$ and $d[v] = \infty$ for $v \neq s$. Then, the node u with the lowest accumulated cost is iteratively selected, and its edges are relaxed by updating the distances of its neighboring nodes if a lower-cost path is found. This process continues until all nodes are processed or the destination is reached.

To optimize the selection of the node with the lowest cost, a priority queue—such as a binary heap or Fibonacci heap—is used, achieving a computational complexity of $O((|V| + |E|) \log |V|)$.

2) A* ALGORITHM

The A* algorithm, introduced by Hart et al. in [29], is a heuristic extension of Dijkstra's algorithm that incorporates an estimated cost function to guide the search more efficiently toward the goal. Currently, A* has numerous extensions, including D* [30], which is designed for dynamic environments. However, A* remains widely used in global planning due to its efficiency in static environments and its lower computational cost compared to D* [31]. Unlike Dijkstra, which only considers the accumulated cost, A* employs a heuristic function $h(v)$ to estimate the remaining cost to the goal, defining its evaluation function as in (2):

$$f(v) = g(v) + h(v) \quad (2)$$

where $g(v)$ represents the accumulated cost from the start node s to v , and $h(v)$ is the heuristic estimate of the cost

from v to the goal. The heuristic function must be admissible, meaning it never overestimates the actual cost ($h(v) \leq h^*(v)$, where $h^*(v)$ is the true cost), and consistent (monotonic), satisfying $h(v) \leq w(v, u) + h(u)$ for any edge (v, u) .

The choice of the heuristic function $h(v)$ significantly influences the performance of the algorithm. In grid-based environments, a common heuristic is the Euclidean distance:

$$h(v) = \sqrt{(x_v - x_g)^2 + (y_v - y_g)^2} \quad (3)$$

where (x_v, y_v) and (x_g, y_g) represent the coordinates of node v and the goal g , respectively. When $h(v)$ is admissible, A* guarantees finding an optimal solution.

3) RRT* ALGORITHM

The Optimal Rapidly-exploring Random Tree (RRT*) algorithm, unlike A* and Dijkstra, which are graph-based methods, is a sampling-based planning algorithm. RRT* is an asymptotically optimal extension of RRT [32]. In its basic form, it incrementally constructs a tree of feasible trajectories rooted at the initial condition.

Proposed by Karaman and Frazzoli in [33], RRT* is derived from the Rapidly-exploring Random Graph (RRG) algorithm, which introduces denser connections compared to RRT. RRT* removes these redundant edges, forming a graph structure.

To apply RRT*, the configuration space C is defined, with obstacle-free regions $C_{\text{free}} \subset C$. The objective is to find a continuous trajectory $\sigma : [0, 1] \rightarrow C_{\text{free}}$ that connects an initial configuration $x_{\text{init}} \in C_{\text{free}}$ with a goal region $C_{\text{goal}} \subset C_{\text{free}}$, minimizing a cost function $J(\sigma)$, typically the Euclidean distance.

The algorithm follows an iterative process in which a random sample point x_{rand} is generated, and the nearest node in the tree x_{nearest} is found. From this node, an expansion step toward x_{rand} generates x_{new} , provided the connection is collision-free. A key aspect of RRT* is the tree restructuring process to improve the obtained solution through two steps: optimal connection of the new node and tree rewiring.

In the first phase, a set of neighboring nodes X_{near} within an adaptive search radius is determined. For each node x_{near} , it is evaluated whether connecting x_{new} instead of x_{nearest} reduces the accumulated cost from the root, selecting the connection with the lowest cost as given in Equation (4):

$$c_{\min} = \min \left\{ \begin{array}{l} \text{Cost}(x_{\text{nearest}}) + c(\text{Line}(x_{\text{nearest}}, x_{\text{new}})), \\ \text{Cost}(x_{\text{near}}) + c(\text{Line}(x_{\text{near}}, x_{\text{new}})) \end{array} \right\} \quad (4)$$

In the second phase, it is verified whether connecting x_{near} to x_{new} reduces its accumulated cost. If so, its parent node in the tree structure is updated, globally optimizing the trajectory. This mechanism allows the obtained solution to asymptotically converge to the optimum with sufficient iterations.

B. EXPERIMENTAL SETUP

Here, we present the autonomous navigation system implemented for our proposed TSL-2D mechanism in [11],

TABLE 1. Hardware characteristics and ROS 2 topics with message types and rates.

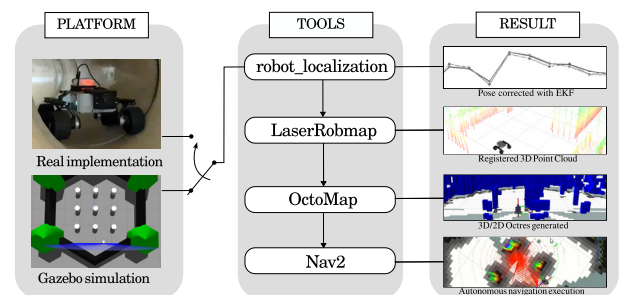
Hardware	Characteristic	Node Name	Topic Name	Message Type	Rate (Hz)
Leo Rover	433 × 447 × 249 mm 6.5 kg weight 5000 mAh Li-ion battery Source 11.1 V DC 5 MPx camera 170°FOV 2.4 GHz WiFi AP ROS & ROS 2	firmware	/firmware/battery	std_msgs/msg/Float32	9.8
			/firmware/battery_averaged	std_msgs/msg/Float32	9.9
			/firmware/imu	leo_msgs/msg/Imu	98.9
			/firmware/wheel_odom	leo_msgs/msg/WheelOdom	20.0
			/firmware/wheel_states	leo_msgs/msg/WheelStates	20.0
		firmware_msg_converter	/imu/data_raw	sensor_msgs/msg/Imu	100.3
			/joint_states	sensor_msgs/msg/JointState	18.2
			/odometry_merged	nav_msgs/msg/Odometry	100.0
			/wheel_odom_with_covariance	nav_msgs/msg/Odometry	20.0
			/robot_state_publisher	tf2_msgs/msg/TFMessage	20.0
Scanner S3	Scanning range ≤ 40m Angle range 0°- 360° Angle Res. ≥ 0.1125° Sampling rate ≤ 32 KHz Light resistance ≤ 80K lux	slidar_node	/scan	sensor_msgs/msg/LaserScan	10.0
VN-300	24 × 22 × 3 mm 3.2-5.5 V power supply 3-axis accelerometers 3-axis gyros 3-axis magnetometer 1 barometric pressure sensor 2 × 50-channel L1 GPS RS-232 or USB cable.	vn_sensor_msgs	/vectomav/gnss	sensor_msgs/msg/NavSatFix	9.9
			/vectomav/imu	sensor_msgs/msg/Imu	10.0
			/vectomav/imu_uncompensated	sensor_msgs/msg/Imu	10.0
			/vectomav/magnetic	sensor_msgs/msg/MagneticField	9.9
			/vectomav/pose	geometry_msgs/msg/PoseWithCovarianceStamped	10.0
			/vectomav/pressure	sensor_msgs/msg/FluidPressure	9.8
			/vectomav/temperature	sensor_msgs/msg/Temperature	9.9
			/vectomav/time_gps	sensor_msgs/msg/TimeReference	9.9
			/vectomav/time_pps	sensor_msgs/msg/TimeReference	9.7
			/vectomav/time_startup	sensor_msgs/msg/TimeReference	9.9
			/vectomav/time_syncin	sensor_msgs/msg/TimeReference	9.8
			/vectomav/velocity_body	geometry_msgs/msg/TwistWithCovarianceStamped	9.9
		laserscan_projector	/cloud	sensor_msgs/msg/PointCloud2	5.0
		ekf_filter_node	/odometry/filtered	nav_msgs/msg/Odometry	5.0
		octomap_server	/free_cells_vis_array	visualization_msgs/msg/MarkerArray	3.0
			/occupied_cells_vis_array	visualization_msgs/msg/MarkerArray	3.0
			/projected_map	nav_msgs/msg/OccupancyGrid	3.0
HP ZBook	Intel Core Ultra 7-155H 4.8 GHz 16 Cores CPU 16 GB DDR5 RAM NVIDIA RTX 500 Ada 4 GB GDDR6 VRAM Ubuntu 22.04 LTS ROS & ROS 2	static_transform_publisher	/tf	tf2_msgs/msg/TFMessage	20.0
		rviz	/goal_pose	geometry_msgs/msg/Pose	–
		controller_server	/cmd_vel	geometry_msgs/msg/Twist	20.0
			/local_plan	nav_msgs/msg/Path	–
		bt_navigator	/navigate_to_pose/_action/feedback	nav2_msgs/action/NavigateToPose:Feedback	–
			/navigate_to_pose/_action/status	action_msgs/msg/GoalStatusArray	–
		planner_server	/plan	nav_msgs/msg/Path	–
			/global_costmap/transition_event	lifecycle_msgs/msg/TransitionEvent	–
Turtlebot3 Simulator	Burger-based model Simulation in Gazebo Physics-based dynamics ROS 2 compatibility LiDAR, IMU, encoders Differential-drive	robot_state_publisher	/robot_description	std_msgs/msg/String	0.0
			/tf	tf2_msgs/msg/TFMessage	48.7
		turtlebot3_diff_drive	/tf_static	tf2_msgs/msg/TFMessage	20.0
			/odom	nav_msgs/msg/Odometry	29.3
		turtlebot3_imu	/tf	tf2_msgs/msg/TFMessage	48.7
			/imu	sensor_msgs/msg/Imu	199.6
			/joint_states	sensor_msgs/msg/JointState	29.3
		turtlebot3_laserscan	/scan	sensor_msgs/msg/LaserScan	5.0

detailing the integration of its hardware and software components. Figure 3 illustrates the ROS 2-based system workflow, organized into three main sections. The PLATFORM block corresponds to the Data Source Layer, the TOOLS block corresponding to Processing Layer; and finally, the RESULT block highlights how each tool incrementally contributes to achieving autonomous navigation, corresponding to the Application Layer.

1) DATA SOURCE LAYER

This layer specifically refers to the hardware setup that provides the necessary data, as detailed in Table 1.

This table specifies the hardware model, corresponding ROS topics, message types and rate, available for system integration. However, the hardware setup can be substituted with the TurtleBot3 simulation in Gazebo, requiring only minor modifications. These adjustments involve configuring the scanner's placement within

**FIGURE 3.** Workflow of the autonomous navigation system for the TSL-2D mechanism.

the turtlebot3_description³ and turtlebot3_gazebo⁴ packages to ensure compatibility with the proposed navigation framework.

³http://wiki.ros.org/turtlebot3_description

⁴http://wiki.ros.org/turtlebot3_gazebo

2) PROCESSING LAYER

This module defines the essential ROS 2 packages required for system execution. The `robot_localization`⁵ package employs an Extended Kalman Filter (EKF) to estimate the robot's state using encoder-based odometry and IMU data. The `LaserRobMap`⁶ package processes laser scans to generate 3D point clouds for the TSL-2D mechanism, where range and resolution are determined by its configuration parameters [34]. Table 2 presents the specific values used for both simulation and real-world implementation. The parameter values have been selected according to the tool documentation and others on an experimental basis.

TABLE 2. Configuration parameters for `LaserRobMap` and `OctoMap` in simulation and real-world implementation.

Tools	Params	Platform		Unit
		Simulation	Real-World	
LaserRobMap	frequency	4	5	Hz
	angle_min	0	-1.57	rad
	angle_max	6.28	1.57	rad
	angle_res	0.05	0.01	rad
	step_line	0.01	0.06	m
	step_rot	0.01	0.035	rad
OctoMap	resolution	0.05	0.1	m
	frame_id	odom		-
	max_range	3	10	m
	point_cloud_max_z	3.5	0.5	m
	point_cloud_min_z	0.15		m

The `OctoMap`⁷ package constructs a probabilistic occupancy map from point cloud data. It efficiently integrates sensor measurements while accounting for noise and outliers, using an octree structure to optimize memory usage and computational performance. Its multi-resolution mapping capability allows for scalable representations, while its ray-casting method accurately distinguishes between free, occupied, and unknown voxels, enhancing spatial awareness for robotic navigation.

The Nav2 stack is a modular and flexible navigation framework for ROS 2 that enables autonomous robot movement in dynamic environments. It provides essential functionalities such as localization, path planning, behavior coordination, and motion control. In our implementation, Nav2 is not used for localization, its core components include the `controller_server` for real-time trajectory execution, the `planner_server` for global path computation, the `bt_navigator` for high-level decision-making using behavior trees, and the `lifecycle_manager` for state management and system orchestration.

Figure 4 also details the high-level navigation system used in this study. The SLAM component relies on localization based on encoder-driven odometry and mapping through scanning projection. Unlike traditional robotic navigation setups, LiDAR odometry with Monte Carlo localization is not feasible due to the non-conventional mounting of the

scanner, as proposed in this study. Consequently, SLAM accuracy depends on EKF-based state estimation, while the scanner is utilized directly for 3D point cloud generation, the resulting 3D point clouds are then projected into a 2D probabilistic occupancy map. This map is incrementally built as the robot rotates around its own axis and serves as the foundation for autonomous navigation. The NAVIGATION component utilizes the 2D grid map to compute motion plans toward a specified 2D Goal Pose, which serves as the basis for navigating to user-defined targets. Although the system supports the integration of waypoint selection or frontier detection algorithms, such functionalities are not evaluated in this study. Instead, the 2D Goal Pose is manually assigned through RViz, based on the perceived environment (support for autonomous waypoint selection is considered for future work). On the left side of the diagram, the minimal data provided by the TSL-2D robot is depicted, including pose, orientation, and laser scans.

III. RESULTS

A. AUTONOMOUS NAVIGATION TEST RESULTS

This section presents two experiments designed to evaluate the performance of global path planning algorithms and demonstrate feasibility of autonomous navigation with three-dimensional SLAM in the TSL-2D configuration. To this end, the first experiment is conducted in the Gazebo simulator, while the second is performed in a real-world environment within our laboratory, using a physical robot as illustrated in Figure 5.

For each experiment, multiple trials were conducted to compare the trajectories generated by the global planning algorithms Dijkstra, A*, and RRT*, as described in Section II-A. Additionally, Table 3 summarizes the parameters used for teleoperation, the Nav2 configuration, and the coordinates of the waypoints in each test environment. All tests use the defined waypoints to ensure that the conditions are the same for each test. It is important to note that most Nav2 nodes retain their default configuration values.

The experimental procedure follows a structured sequence, as shown in the video,⁸ and is consistently repeated for each algorithm. A total of nine tests were conducted, comprising three trials for each algorithm:

- **(S.1) System Initialization:** The process begins with the activation of the test environment. The simulator or physical robot is powered on, and the corresponding nodes for data acquisition and processing layers are launched.
- **(S.2) Environment Recognition:** Manual control via teleoperation is enabled, allowing the robot to rotate around its axis to explore the surroundings. If nearby obstacles are detected, the robot maneuvers around them to enhance spatial perception. This phase continues

⁵https://github.com/cra-ros-pkg/robot_localization

⁶https://github.com/FredMSaico/Laser_RobMap

⁷<https://github.com/OctoMap>

⁸<https://youtu.be/BVbBBFpAmoU>

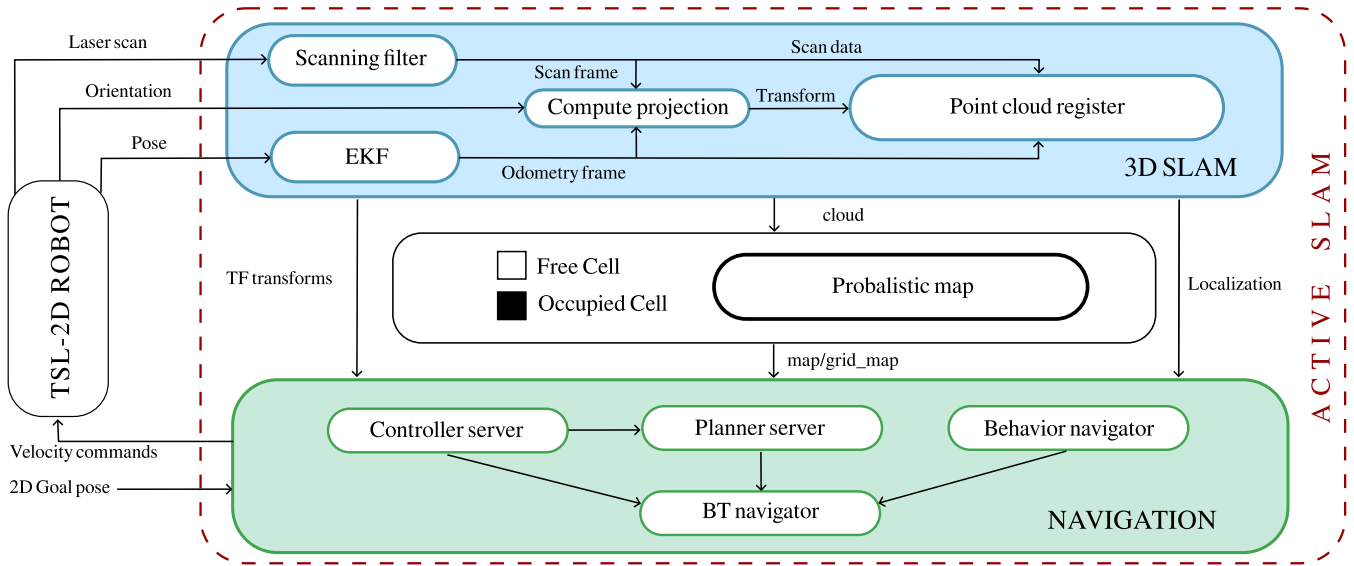


FIGURE 4. High-level diagram of the three-dimensional SLAM-based robotic navigation system used in this study.

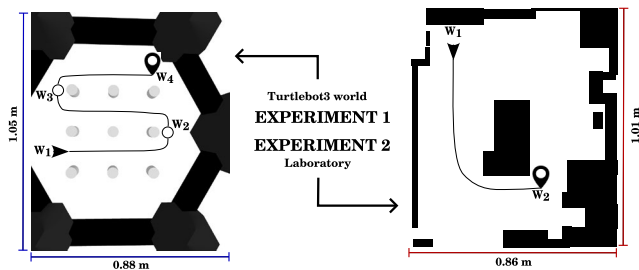


FIGURE 5. Diagram of the experimental scenarios in simulated environment (Turtlebot3 world) and actual environment (Laboratory) with real dimensions.

until a sufficient understanding of the environment is achieved.

- **(S.3) Environment Evaluation:** Once the initial exploration is completed, the system analyzes the acquired data and verifies that the environmental representation is adequate for path planning.
- **(S.4) Waypoint Assignment:** With the environment information validated, reference points for autonomous navigation are established. In the simulation, four waypoints (W_1, W_2, W_3, W_4) are defined, whereas only two (W_1, W_2) are used in the real-world experiment.
- **(S.5) Navigation Execution:** The global trajectory is generated and integrated with the navigation software controllers. The robot begins its movement, following the planned path until the final waypoint is reached, marking the completion of the process.

The results of the experiment conducted in the Gazebo simulator are presented in Figure 6, where the trajectories obtained in a 2D plane for each algorithm are shown. It can be observed that all three methods successfully generate feasible

paths that avoid obstacles in the environment, with some differences between them. The dashed lines in different colors represent the paths generated in each of the trials conducted. Additionally, a black line has been included to indicate the average trajectory, along with a shaded region representing the variance of the trajectories among the three trials.

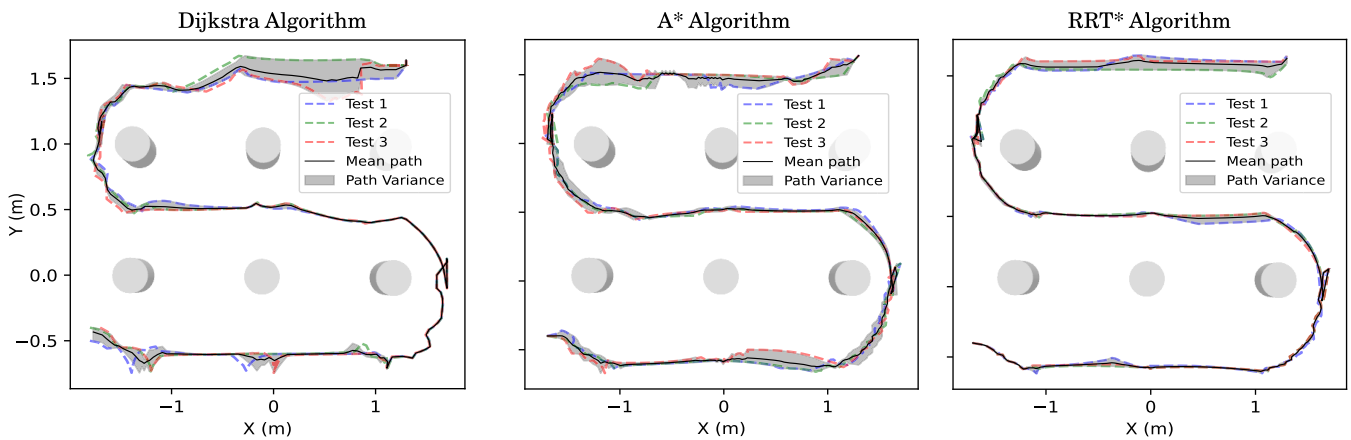
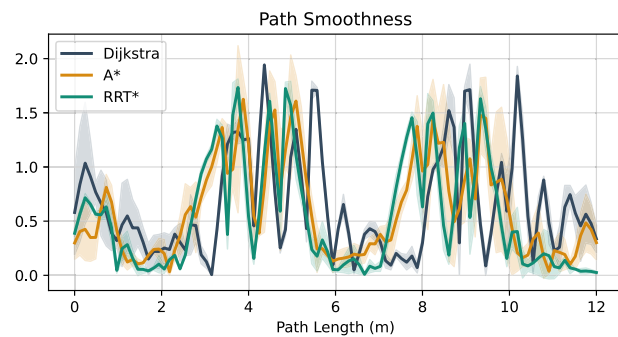
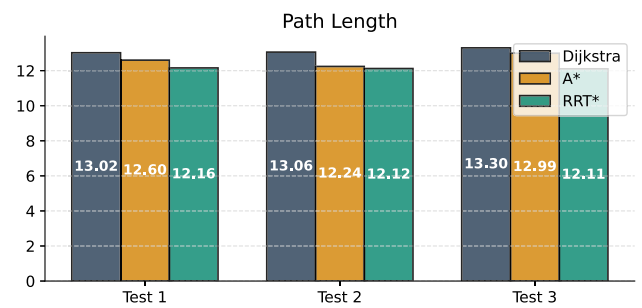
Figure 7 illustrates the variation in trajectory smoothness as a function of the traveled distance. Smoothness is quantified using the curvature magnitude, obtained as the absolute value of the second derivative of the smoothed trajectory. This is computed through interpolation over a uniform distance scale followed by a Savitzky-Golay filtering, which reduces noise in the data without significantly distorting the trajectory structure. The comparison between algorithms reveals that Dijkstra generates paths with more abrupt direction changes, which may negatively impact robot navigation by introducing sharp turns. In contrast, A* and RRT* produce similar trajectories that are smoother and exhibit lower curvature variability, facilitating more stable motion.

Additionally, Figure 8 compares the lengths of the paths generated by each algorithm in each trial. The specific length values are displayed on each bar, allowing for a quantitative analysis of the performance of each method. It is observed that Dijkstra tends to generate longer paths compared to A* and RRT*, with an average length of 13.13 ± 0.17 . The A* algorithm achieves a better balance between path length and efficiency, with an average of 12.61 ± 0.38 . On the other hand, RRT* produces the shortest trajectory, with an average of 12.13 ± 0.03 , representing a 7.62% reduction in length compared to Dijkstra.

The accuracy of TSL-2D approach, has been validated in [11], and this study has demonstrated its applicability in autonomous navigation and 3D mapping, as shown in Figure 9. The proposed implementation is highly versatile

TABLE 3. Teleoperation setup, Nav2 parameters and Waypoint values.

Params	Simulated environment	Actual Environment	Unit
Teleoperation			
linear_speed	0.215	0.06	m/s
angular_speed	0.169	0.06	rad/s
map_time	29.5	46.0	s
Nav2 (Controller_server)			
xy_goal_tolerance	0.1	0.1	m
max_vel_x	0.26	0.06	m/s
max_vel_theta	1.0	0.06	rad/s
navigation_time	58.5	151.7	s
Nav2 (Local_costmap, Global_costmap)			
resolution	0.05	0.05	m
robot_radius	0.15	0.25	m
Waypoints (x, y)			
W_1	(-1.5765, -0.5770)	(1.6985, 0.1254)	m
W_2	(1.6985, 0.1254)	(-1.6543, 1.2167)	m
W_3	(-1.6543, 1.2167)	-	m
W_4	(1.2994, 1.6401)	-	m

**FIGURE 6.** Path generated for Dijkstra, A*, and RRT* in simulation tests.**FIGURE 7.** Trajectory smoothness as a function of distance traveled for Dijkstra, A*, and RRT* in simulation tests.**FIGURE 8.** Bar chart showing the trajectory lengths generated by Dijkstra, A*, and RRT* in simulation tests.

and suitable for various fields of robotics, enabling the validation of different planning algorithms, including both global and local planners, as well as motion planning methods. Furthermore, future work aims to integrate a stereo camera to enhance environmental perception and optimize waypoint selection, strengthening the system's capability to operate in more complex and dynamic environments.

B. ACCURACY RESULTS AGAINST RGB-D

This section presents the accuracy results for generating 2D projected maps from the point clouds produced by the proposed TSL-2D mechanism, in comparison with the widely used and low-cost RGB-D method. Both methods followed the same experimental procedure described in Section III-A, from (S.1) System Initialization to (S.3) Environment Evaluation, and used identical parameter settings as listed in Table 2.

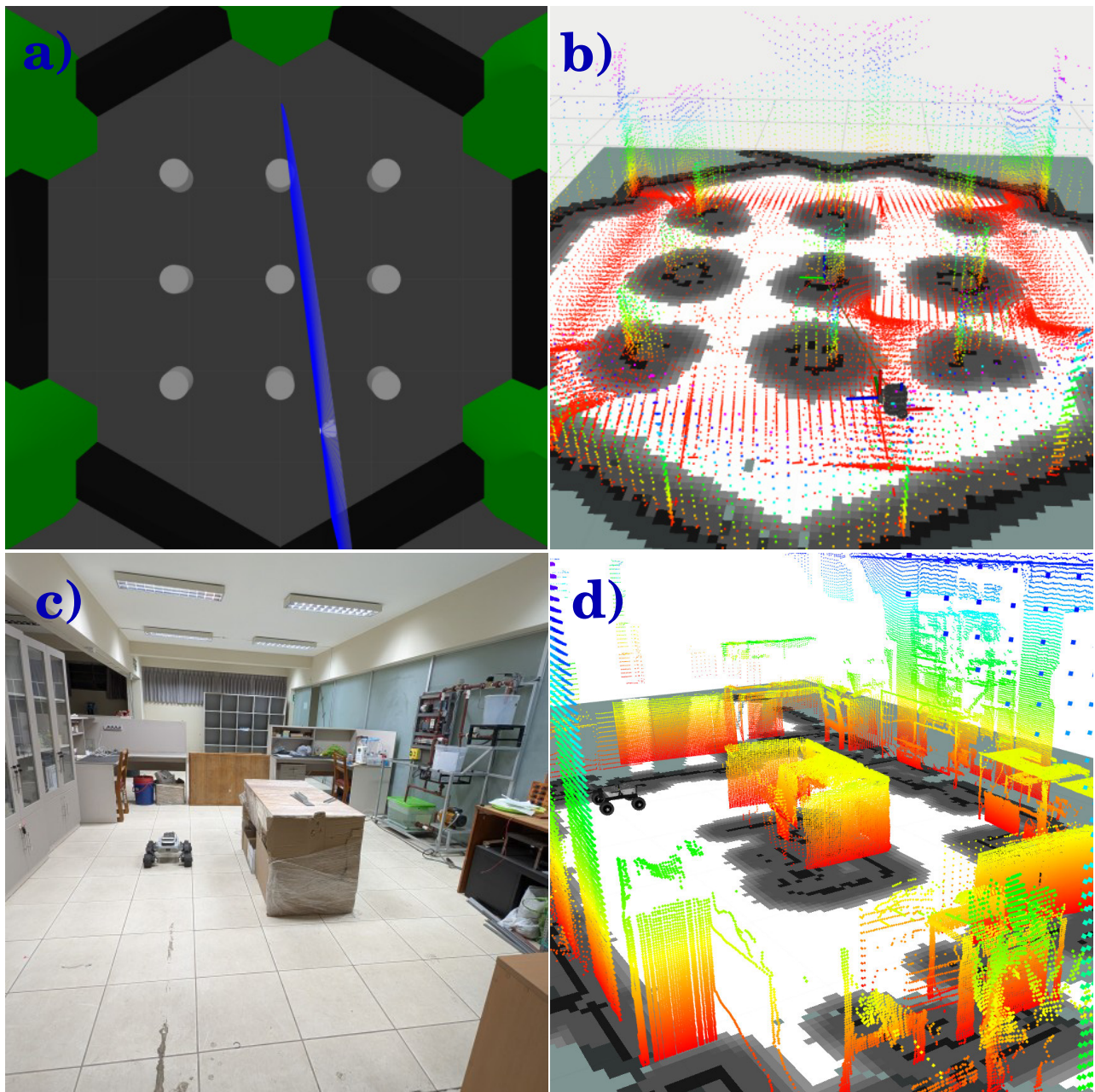


FIGURE 9. Three-dimensional SLAM results in Gazebo simulation and real-world implementation. (a) Simulated environment in Gazebo. (b) Point cloud, 2D map, and global and local map costs in the Gazebo simulation. (c) Photograph of the laboratory environment used for the implementation. (d) Point cloud, 2D map, and global and local map costs in the real-world implementation.

Four trials were conducted for each method, progressively increasing both linear and angular velocity. The results reveal that the point clouds obtained from the RGB-D camera exhibit significant fluctuations, with broad margins around detected objects and considerable noise in free space, as show in Figure 10. This reduces the reliability of map cost estimation and impairs the execution of autonomous navigation tasks. In contrast, the point clouds captured by the TSL-2D mechanism resulted in cleaner and more defined maps, allowing for more accurate costmap calculation

and enabling the robot to follow autonomous trajectories effectively.

Additionally, three points of interest (A, B, and C) were identified across the projected maps, except for those generated in trial M-4, where both methods failed due to high angular velocity. These points were used to quantitatively assess the precision of each method through geometric measurements.

The results demonstrate that the TSL-2D mechanism delivers superior precision across all evaluated metrics.

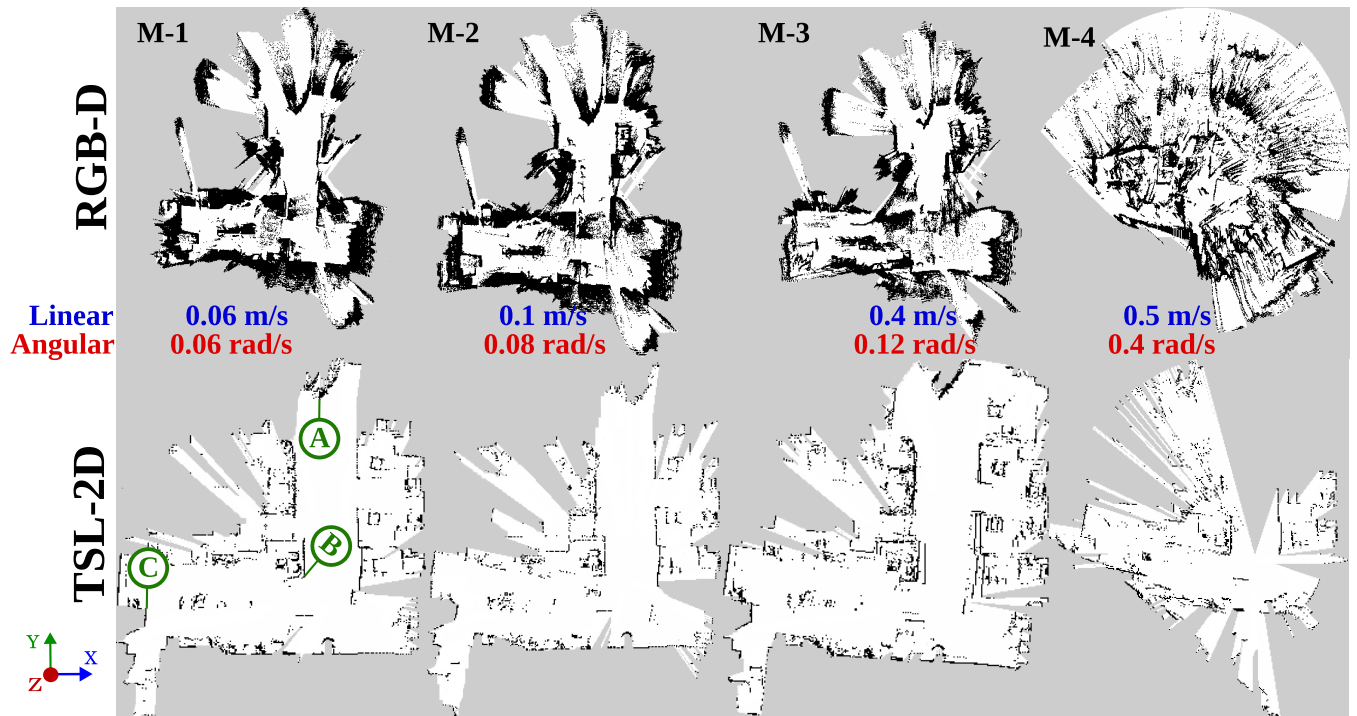


FIGURE 10. Projected 2D maps generated from point clouds using the RGB-D and TSL-2D methods across four experimental trials (M-1 to M-4).

TABLE 4. Comparison of geometric measurements obtained using the RGB-D and TSL-2D methods to evaluate their precision in generating projected 2D maps.

Method	Metric	M-1	M-2	M-3	M-4	Std. Dev.	Unit
RGB-D	A→B Length	7.958	7.550	8.112	N/A	0.285	m
	A→B Angle	-95.73	-96.34	-95.212	N/A	0.566	°
	B→C Length	6.872	6.783	7.395	N/A	0.337	m
	B→C Angle	11.265	11.310	4.869	N/A	3.725	°
	∠ABC	107.311	107.574	96.934	N/A	5.899	°
TSL-2D	A→B Length	7.630	7.690	7.645	N/A	0.030	m
	A→B Angle	-94.606	-96.734	-94.092	N/A	1.400	°
	B→C Length	6.880	6.824	6.899	6.577	0.139	m
	B→C Angle	11.049	11.155	11.030	N/A	0.065	°
	∠ABC	105.438	106.644	105.094	N/A	0.798	°

This is particularly evident in its significantly lower standard deviations for the distances A→B, B→C, and angle $\angle ABC$, when compared to the RGB-D method.

IV. DISCUSSIONS

The results obtained in this study indicate that the TSL-2D mechanism offers advantages for specific applications, but does not constitute a general-purpose solution suitable for all environments. Therefore, this section discusses its characteristics in comparison to the most common alternatives: RGB-D cameras and mechanical 3D LiDAR sensors. Table 5 outlines key differences among TSL-2D, RGB-D, and mechanical 3D LiDAR systems. Based on this comparison, specific scenarios are identified in which the proposed TSL-2D mechanism is more appropriate.

Compared to RGB-D cameras such as the widely used Intel RealSense D455, the TSL-2D system offers notable

advantages in terms of range and cost-efficiency. According to the datasheet, the D455 achieves a depth resolution below 10 mm with a maximum range of up to 6 meters. In contrast, the scanner implemented in the TSL-2D mechanism provides a resolution under 30 mm, with an extended range of up to 40 meters. Furthermore, the overall cost of the TSL-2D setup is approximately five times lower than that of an RGB-D camera, making it a viable alternative for low-budget applications.

Regarding mechanical 3D LiDAR sensors, such as the Velodyne VLP-16—which performs multi-plane scanning (typically 16 channels)—these devices are effective for obstacle perception in unstructured environments. However, they are unable to capture the full volume above and below the sensor, which limits their 3D mapping capabilities in enclosed spaces unless additional rotational mechanisms are integrated to increase vertical coverage [35]. In this context,

TABLE 5. Comparative analysis between TSL-2D, RGB-D cameras, and mechanical 3D LiDARs.

Feature	TSL-2D	RGB-D Camera (e.g., D455)	Mechanical 3D LiDAR (e.g., VLP-16)
Vertical Coverage	Continuous via linear motion	Fixed, narrow field of view	Discrete (e.g., 16 scan planes)
Depth Resolution	<30 mm	<10 mm	<30 mm
Range	Up to 40 m	Up to 6 m	Up to 100 m
Cost	Low (1×)	Medium–High (5× TSL-2D)	High (20× TSL-2D)
Mapping Completeness	High in indoor/static scenes	Low (limited vertical view)	Medium (incomplete top/bottom views)
Sensitivity to Reflective Surfaces	High (prone to errors)	High (Lack of light)	Low
Dependency on Odometry	High	Medium	Low
Suitability for Dynamic Environments	Limited	Moderate	High
Recommended Use Cases	Indoor mapping, low-cost robots	Short-range perception, SLAM	Autonomous vehicles, outdoor mapping

the TSL-2D mechanism enables more complete environmental modeling through continuous vertical exploration. Additionally, by relying on a translational rather than a rotational scanning system, it reduces geometric distortion errors typically accumulated during rotational sweeps.

Despite these advantages, the TSL-2D approach presents notable limitations. First, its accuracy is highly dependent on the quality of the odometry, making it susceptible to cumulative drift over long trajectories—particularly in the absence of loop closure mechanisms or external localization sources [12]. Moreover, since it generates point clouds via fixed vertical scanning combined with the robot’s forward motion, the system’s field of view is primarily constrained to the translational plane. This configuration may miss laterally moving objects or elements located outside the scanning axis in dynamic environments. Additionally, the use of low-cost LiDAR sensors on reflective surfaces, such as glass, or highly diffusive materials, such as white surfaces, can result in erroneous readings or incomplete data [21].

Based on the comparison, specific use cases are identified where the TSL-2D provides a more efficient and cost-effective alternative to RGB-D cameras and mechanical 3D LiDARs. These include static environments such as abandoned tunnels, enclosed corridors, decommissioned factories, deteriorated infrastructure, or contaminated warehouses, where detailed 3D mapping is a priority. In such scenarios, the robot can be teleoperated during exploration, and in case of an emergency—such as loss of communication—autonomous navigation can be activated to return to the origin using the preconstructed map.

V. CONCLUSION

This study has demonstrated that the Translational Single LiDAR (TSL-2D) mechanism is a viable and cost-effective alternative for integrating autonomous navigation and 3D mapping in mobile robots. By using a single vertically mounted 2D LiDAR sensor actuated translationally, the system achieves accurate three-dimensional reconstruction and effective autonomous navigation in static environments. This setup eliminates the need for expensive 3D LiDAR

hardware in specific applications, such as the exploration of abandoned or contaminated infrastructures, where cost reduction and system simplicity are essential.

Developed within the ROS 2 framework and validated through Gazebo simulations and real-world experiments, the system supports the integration and evaluation of global path planning algorithms. For validation purposes, Dijkstra, A*, and RRT* were compared. The comparative analysis showed that all three algorithms are applicable to the proposed configuration, with RRT* generating the smoothest and shortest trajectories in the selected test scenario.

Moreover, the proposed implementation proved to be highly adaptable, allowing the integration and evaluation of various planning and navigation algorithms, including local and motion planners. Future work will focus on enhancing environmental perception through the incorporation of stereo cameras, computational cost on the onboard computer, integrating waypoint selection algorithms, analyzing the effects of path planning algorithms on the final 3D map quality, and evaluating the system in more complex environments.

REFERENCES

- [1] A. R. Khairuddin, M. S. Talib, and H. Haron, “Review on simultaneous localization and mapping (SLAM),” in *Proc. IEEE Int. Conf. Control Syst., Comput. Eng. (ICCSC)*, Nov. 2015, pp. 85–90.
- [2] B. Al-Tawil, A. Candemir, M. Jung, and A. Al-Hamadi, “Mobile robot navigation with enhanced 2D mapping and multi-sensor fusion,” *Sensors*, vol. 25, no. 8, p. 2408, Apr. 2025.
- [3] J. Cheng, L. Zhang, Q. Chen, X. Hu, and J. Cai, “A review of visual SLAM methods for autonomous driving vehicles,” *Eng. Appl. Artif. Intell.*, vol. 114, Sep. 2022, Art. no. 104992.
- [4] Y. Zhang, L. Zhang, Q. Yu, and B. Xing, “Research on the visual SLAM algorithm for unmanned surface vehicles in nearshore dynamic scenarios,” *J. Mar. Sci. Eng.*, vol. 13, no. 4, p. 679, Mar. 2025.
- [5] S. Huai, L. Cao, Y. Zhou, Z. Guo, and J. Gai, “A multi-strategy visual SLAM system for motion blur handling in indoor dynamic environments,” *Sensors*, vol. 25, no. 6, p. 1696, Mar. 2025.
- [6] X. Yue, Y. Zhang, and M. He, “LiDAR-based SLAM for robotic mapping: State of the art and new frontiers,” 2023, *arXiv:2311.00276*.
- [7] J. Jiang, T. Zhang, and K. Li, “LiDAR-based 3D SLAM for autonomous navigation in stacked cage farming houses: An evaluation,” *Comput. Electron. Agricult.*, vol. 230, Mar. 2025, Art. no. 109885.
- [8] D. Tiozzo Fasiolo, L. Scalera, and E. Maset, “Comparing LiDAR and IMU-based SLAM approaches for 3D robotic mapping,” *Robotica*, vol. 41, no. 9, pp. 2588–2604, Sep. 2023.

- [9] C. Pang, L. Zhou, and X. Huang, "A low-cost 3D SLAM system integration of autonomous exploration based on fast-ICP enhanced LiDAR-inertial odometry," *Remote Sens.*, vol. 16, no. 11, p. 1979, May 2024.
- [10] E. Maset, L. Scalera, A. Beinat, D. Visintini, and A. Gasparetto, "Performance investigation and repeatability assessment of a mobile robotic system for 3D mapping," *Robotics*, vol. 11, no. 3, p. 54, Apr. 2022.
- [11] P. R. Yanyachi, A. Mamani Saico, F. Chacon, M. Esquivel, J. C. Cutipa Luque, and D. Yanyachi, "Three-dimensional reconstruction of enclosed environments based on two-dimensional LiDAR: Starting point and future challenges," *IEEE Latin Amer. Trans.*, vol. 22, no. 8, pp. 704–712, Aug. 2024.
- [12] S. Bi, C. Yuan, C. Liu, J. Cheng, W. Wang, and Y. Cai, "A survey of low-cost 3D laser scanning technology," *Appl. Sci.*, vol. 11, no. 9, p. 3938, Apr. 2021.
- [13] J. Baek, "Two-dimensional LiDAR sensor-based three-dimensional point cloud modeling method for identification of anomalies inside tube structures for future hypersonic transportation," *Sensors*, vol. 20, no. 24, p. 7235, Dec. 2020.
- [14] I. Baldwin and P. Newman, "Laser-only road-vehicle localization with dual 2D push-broom LiDARs and 3D priors," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, Oct. 2012, pp. 2490–2497.
- [15] A. Napier, P. Corke, and P. Newman, "Cross-calibration of push-broom 2D LiDARs and cameras in natural scenes," in *Proc. IEEE Int. Conf. Robot. Autom.*, May 2013, pp. 3679–3684.
- [16] A. Wang, C. Li, Y. Liu, Y. Zhuang, C. Bu, and J. Xiao, "Laser-based online sliding-window approach for UAV loop-closure detection in urban environments," *Int. J. Adv. Robotic Syst.*, vol. 13, no. 2, p. 61, Mar. 2016.
- [17] H. Zhu, H. Wang, and S. Sun, "Research on DEM reconstruction based on airborne 2D laser radar," in *Proc. 37th Chin. Control Conf. (CCC)*, Jul. 2018, pp. 8435–8440.
- [18] K. K. Saha, C. Weltzien, and M. Zude-Sasse, "Non-destructive leaf area estimation of tomato using mobile LiDAR laser scanner," in *Proc. IEEE Int. Workshop Metrology Agricult. Forestry (MetroAgriFor)*, Nov. 2021, pp. 187–191.
- [19] C. Hu, C. Yang, K. Li, and J. Zhang, "A forest point cloud real-time reconstruction method with single-line LiDAR based on visual-IMU fusion," *Appl. Sci.*, vol. 12, no. 9, p. 4442, Apr. 2022.
- [20] R. Athavale, D. S. Harish Ram, and B. B. Nair, "Low cost solution for 3D mapping of environment using 1D LiDAR for autonomous navigation," *IOP Conf. Ser., Mater. Sci. Eng.*, vol. 561, no. 1, Oct. 2019, Art. no. 012104.
- [21] M. Yusuf, A. Zaidi, A. Haleem, S. Bahl, M. Javaid, S. Bala Garg, and J. Garg, "IoT-based low-cost 3D mapping using 2D LiDAR for different materials," *Mater. Today, Proc.*, vol. 57, pp. 942–947, Jan. 2022.
- [22] N. Sariff and N. Buniyamin, "An overview of autonomous mobile robot path planning algorithms," in *Proc. 4th Student Conf. Res. Develop.*, Jun. 2006, pp. 183–188.
- [23] Z. Tang and H. Ma, "An overview of path planning algorithms," *IOP Conf. Ser., Earth Environ. Sci.*, vol. 804, no. 2, Jul. 2021, Art. no. 022024.
- [24] B. Zhu, C. Li, L. Song, Y. Song, and Y. Li, "A algorithm of global path planning based on the grid map and V-graph environmental model for the mobile robot," in *Proc. Chin. Autom. Congr. (CAC)*, Oct. 2017, pp. 4973–4977.
- [25] A. Choudhary, "Sampling-based path planning algorithms: A survey," 2023, *arXiv:2304.14839*.
- [26] H. Sang, Y. You, X. Sun, Y. Zhou, and F. Liu, "The hybrid path planning algorithm based on improved A* and artificial potential field for unmanned surface vehicle formations," *Ocean Eng.*, vol. 223, Mar. 2021, Art. no. 108709.
- [27] L. Zhong, X. Tang, L. Liu, G. Yang, K. Guo, and H. Sun, "Hybrid path planning algorithm based on improved dynamic window approach," in *Proc. IEEE 2nd Int. Conf. Big Data, Artif. Intell. Internet Things Eng. (ICBAIE)*, Mar. 2021, pp. 57–62.
- [28] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Math.*, vol. 1, no. 1, pp. 269–271, Dec. 1959.
- [29] P. Hart, N. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Syst. Sci. Cybern.*, vol. SSC-4, no. 2, pp. 100–107, Feb. 1968.
- [30] D. Ferguson and A. Stentz, "The delayed D algorithm for efficient path replanning," in *Proc. IEEE Int. Conf. Robot. Autom.*, Jun. 2005, pp. 2045–2050.
- [31] K. Karur, N. Sharma, C. Dharmatti, and J. E. Siegel, "A survey of path planning algorithms for mobile robots," *Vehicles*, vol. 3, no. 3, pp. 448–468, Aug. 2021.
- [32] S. M. LaValle and J. James J. Kuffner, *Rapidly-Exploring Random Trees: Progress and Prospects*, 1st ed., Boca Raton, FL, USA: CRC Press, 2001, p. 5.
- [33] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *Int. J. Robot. Res.*, vol. 30, no. 7, pp. 846–894, Jun. 2011.
- [34] D. D. Yanyachi, Y. H. Anco-Aguero, G. A. Echaiz, M. A. Esquivel, A. Mamani-Saico, and P. R. Yanyachi, "Laser_RobMap: An open source ROS2 compatible tool for 3D mapping using a mobile robot and 2D LiDAR," *SoftwareX*, vol. 30, May 2025, Art. no. 102142.
- [35] L. Yan, J. Dai, Y. Zhao, and C. Chen, "Real-time 3D mapping in complex environments using a spinning actuated LiDAR system," *Remote Sens.*, vol. 15, no. 4, p. 963, Feb. 2023.



MARJORIE A. LEÓN RAMOS was born in Arequipa, Peru. She received the degree in electronic engineering from the Universidad Nacional de San Agustín de Arequipa, Peru, in 2019, and the degree in industrial engineering from the Universidad Tecnológica del Perú, in 2023. She is currently an Engineer in the mining sector. Her research interests include TICS, robotics, and data mining.



RAUL PAOLO RIVERA VERA was born in Arequipa, Peru. He received the degree in electronic engineering and the M.Sc. degree in electronics with a specialization in automation and control from the Universidad Nacional de San Agustín de Arequipa (UNSA). He is currently an Automation and Technology Engineer with Komatsu-Mitsui Maquinarias Peru. His research interests include autonomous vehicles, robotics, and intelligent control systems.



DANIEL D. YANYACHI (Member, IEEE) was born in Peru. He received the M.Sc. and Ph.D. degrees in electrical engineering from the Leningrad Polytechnic Institute. He has lectured for many years as a Full Professor with the Electronic Engineering Department, Universidad Nacional de San Agustín de Arequipa, Peru. His research interests include processing control systems, mining, manufacturing, and complex and advanced control systems.



ALFREDO MAMANI SAICO (Member, IEEE) was born in Cusco, Peru, in 2000. He received the degree in electronic engineering from the Universidad Nacional de San Agustín de Arequipa (UNSA). He is currently a Junior Researcher with the Instituto Astronómico y Aeroespacial Pedro Paulet (IAAPP), UNSA. His research interests include robotics, distributed systems, and nanosatellite development.



JUAN CARLOS CUTIPA LUQUE (Member, IEEE) was born in Arequipa, Peru. He received the degree in electronic engineering from the Universidad Nacional de San Agustín de Arequipa, Peru, in 2004, and the M.Sc. and Ph.D. degrees in mechanical engineering from the University of São Paulo, Brazil, in 2007 and 2012, respectively. He is currently a Professor with the Electronic Engineering Department, Universidad Nacional de San Agustín de Arequipa. His research interests include advanced control systems for autonomous underwater vehicles (AUVs), remote-operated submersible vehicles (ROVs), unmanned surface vehicles (USVs), and electric vehicles.



GERMAN ALBERTO ECHAIZ ESPINOZA was born in Peru. He received the Bachelor of Science (B.Sc.) degree in electrical engineering from the National University of Engineering (UNI), Lima, in 1980, and the Master of Science (M.Sc.) degree in electronic engineering with a specialization in instrumentation and automation and the Doctor of Science (Ph.D.) degree in energy engineering from the National University of San Agustín de Arequipa (UNSA), Peru, in 2003 and 2013, respectively. In 1984, he was awarded the professional title of Electronic Engineer from UNI.



PABLO R. YANYACHI (Senior Member, IEEE) received the Master of Science degree in automatic control from the Polytechnic Institute of Leningrad, and the Ph.D. degree in electrical engineering from the Polytechnic School, University of São Paulo, Brazil. He is a Professor of the Academic Department of Electronic Engineering, National University of San Agustín-Arequipa (UNSA). He is the Station Manager of the NASA Laser Tracking Station TLRS-3, Arequipa, Peru. He is the Director of the Instituto de Investigación Astronómico y Aeroespacial Pedro Paulet (IAAPP), UNSA.

...